

Clickstream Pipeline

Real-Time Data Engineering on AWS

Overview

This project simulates a real-world clickstream tracking system (like an e-commerce or SaaS platform), where user interactions are captured, processed in real time, and made available for analytics. It demonstrates key data engineering concepts: event-driven architecture, real-time stream processing, schema-on-read analytics, data lake design using columnar storage (Parquet), and serverless infrastructure.

Architecture

- Producer (Local App)
- Kinesis Data Streams (Ingestion)
- AWS Lambda (Stream Processing)
- Kinesis Data Firehose (Buffer + Parquet Conversion)
- Amazon S3 (Data Lake - Processed Layer)
- AWS Glue Crawler (Metadata Catalog)
- Amazon Athena (Analytics)

Key Features

- Real-time ingestion using Kinesis Data Streams
- Serverless processing with AWS Lambda
- Automatic JSON → Parquet conversion via Firehose
- Optimized data lake storage in Amazon S3
- Schema discovery using Glue Crawler
- SQL-based analytics with Athena
- Fully event-driven pipeline (no batch jobs required)

Tech Stack

Category	Technology
Language	Python
Streaming	AWS Kinesis Data Streams
Processing	AWS Lambda
Delivery & Transform	Kinesis Data Firehose
Storage	Amazon S3

Catalog	AWS Glue Crawler
Query Engine	Amazon Athena
Format	Apache Parquet

Data Flow Explained

1. Producer Layer

A Python script simulates user behavior by generating clickstream events.

```
{"user_id": "user2", "action": "click", "timestamp": 1710000000}
```

2. Streaming Layer (Kinesis Data Streams)

Receives high-throughput event data. Ensures durability and scalability. Acts as the backbone of the pipeline.

3. Processing Layer (AWS Lambda)

Triggered automatically by Kinesis. Transforms data by decoding records, enriching with processed_at timestamp, and forwarding to Firehose.

4. Delivery Layer (Kinesis Data Firehose)

Firehose handles buffering incoming data, converting JSON → Apache Parquet, and writing compressed, analytics-ready files to S3.

5. Storage Layer (Amazon S3 Data Lake)

Data is stored in Parquet format for faster query performance, reduced storage cost, and columnar access patterns.

6. Catalog Layer (Glue Crawler)

Automatically scans S3, infers schema, and creates tables in the Glue Data Catalog.

7. Analytics Layer (Amazon Athena)

Run SQL queries directly on S3 without managing infrastructure.

Setup & Execution

1. Configure AWS CLI

```
aws configure
```

2. Install Dependencies

```
pip install boto3
```

3. Start the Producer

```
python producer.py
```

4. Run Glue Crawler

Navigate to AWS Glue Console and run: clickstream-crawler. A table will be created in your database.

5. Query in Athena

```
SELECT action, COUNT(*) AS total_events, COUNT(DISTINCT user_id) AS unique_users  
FROM clickstream_db.events GROUP BY action ORDER BY total_events DESC;
```

Design Decisions

Why Firehose for Transformation?

Using Firehose for Parquet conversion eliminates the need for batch ETL jobs, reduces operational overhead, and provides near real-time delivery.

Why Parquet?

Columnar format enables faster queries in Athena, compression lowers S3 storage costs, and it is optimised for analytics workloads.

Why Serverless?

No infrastructure management, auto-scaling, and a pay-per-use cost model.

Challenges & Solutions

- **IAM Permission Errors:** Fixed by attaching appropriate policies to Lambda for Kinesis and Firehose access.
- **Data Not Appearing in S3:** Resolved by configuring Firehose with S3 as the primary destination.
- **Stream Name Mismatch:** Ensured producer and AWS resources used identical names.
- **Athena Query Errors (Table Naming):** Handled edge cases like numeric table names using proper quoting.

Future Improvements

- Add Amazon OpenSearch for real-time search & dashboards
- Introduce EventBridge for event routing (fan-out architecture)
- Implement partitioning strategy in S3 (e.g., by date/hour)
- Add data quality checks in Lambda
- Build a real-time dashboard (e.g., QuickSight)